

LAB: GIAO TIẾP UART – RS232

1. MỤC ĐÍCH

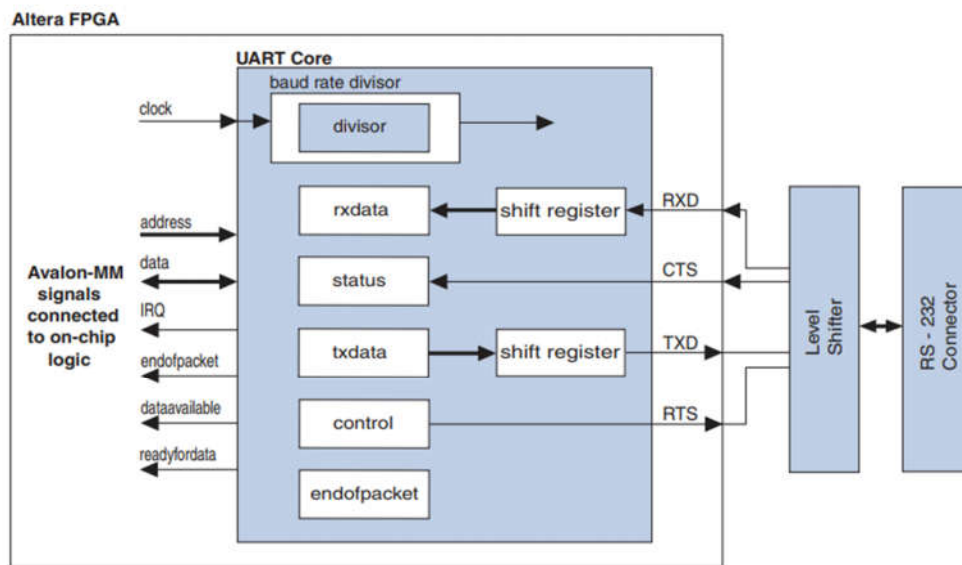
Thông qua bài thực hành này, sinh viên sẽ hiểu rõ:

- Thực hiện truyền nhận data thông qua SerialPort – RS232

2. TỔNG QUAN

Sơ đồ khối UART, với các Input/Output như sau:

Figure 6–1. Block Diagram of the UART Core in a Typical System



Trong đó, chúng ta sẽ truyền data qua chân **Output TXD**, và nhận data tại chân **Input RXD**.

❖ Bảng thanh ghi của lõi UART:

Table 7–4. UART Core Register Map

Offset	Register Name	R/W	Description/Register Bits													
			15:13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	rxdata	RO	Reserved					(1)	(1)	Receive Data						
1	txdata	WO	Reserved					(1)	(1)	Transmit Data						
2	status (2)	RW	Reserved	eop	cts	dcts	(1)	e	rrdy	trdy	tmt	toe	roe	brk	fe	pe
3	control	RW	Reserved	ieop	rts	idcts	trbk	ie	irrdy	itrdy	itmt	itoe	iroe	ibrk	ife	ipe
4	divisor (3)	RW	Baud Rate Divisor													
5	endof-packet (3)	RW	Reserved					(1)	(1)	End-of-Packet Value						

❖ Thanh ghi rxdata:

- Thanh ghi **rxdata** để lưu trữ dữ liệu nhận được qua chân **RXD**. Khi một ký tự mới được nhận

vào **RXD**, nó sẽ được chuyển vào thanh ghi **rxdata** và bit **rrdy** của thanh ghi **status** được set lên **1**. Bit **rrdy** của thanh ghi status được đặt thành **0** khi thanh ghi **rxdata** được đọc.

- Nếu một ký tự được chuyển vào thanh ghi **rxdata** trong khi bit **rrdy** đang ở trạng thái là **1** (nói cách khác, ký tự trước đó không được đọc), thì xảy ra lỗi tràn bộ thu và bit **roe** của thanh ghi **status** sẽ được set **1**.
- Các ký tự mới luôn được chuyển vào thanh ghi rxdata, bất kể ký tự trước đó đã được đọc hay chưa.
- Việc ghi dữ liệu vào thanh ghi rxdata không có tác dụng.

❖ **Thanh ghi txdata:**

- Các ký tự data cần truyền sẽ được ghi vào thanh ghi **txdata**. Không nên ghi ký tự vào **txdata** cho đến khi bộ truyền sẵn sàng cho một ký tự mới, được biểu thị bằng bit **TRDY** trong thanh ghi **status**. Bit **TRDY** được đặt thành **0** khi một ký tự được ghi vào thanh ghi **txdata**. Bit **TRDY** được đặt thành **1** khi ký tự được chuyển từ thanh ghi **txdata** sang thanh ghi dịch chuyển của bộ truyền. Nếu một ký tự được ghi vào thanh ghi **txdata** khi **TRDY** là **0**, kết quả là không được ghi.
- Việc read thanh ghi **txdata** sẽ trả về một giá trị undefined.

❖ **Thanh ghi status:**

- Thanh ghi status bao gồm các bit riêng lẻ biểu thị các điều kiện cụ thể bên trong lõi UART. Mỗi bit trạng thái được liên kết với một bit bật/tắt ngắt tương ứng trong thanh ghi **control**.
- Thanh ghi **status** có thể được đọc bất cứ lúc nào. Việc đọc thanh ghi sẽ không làm thay đổi giá trị của bất kỳ bit nào. Ghi giá trị **0** vào thanh ghi status sẽ xóa các bit DCTS, E, TOE, ROE, BRK, FE và PE.

Bit	Name	Access	Description
0	PE	Read/Clear	Parity error. The PE bit is set to 1 when the core receives a character with an incorrect parity bit.
1	FE	Read/Clear	Framing error. The FE bit is set to 1 when the core receives a character with an incorrect stop bit.
2	BRK	Read/Clear	Break detect. The receiver logic detects a break when the RXD pin is held low 0 continuously for longer than a full-character time.
3	ROE	Read/Clear	Receive overrun error. A receive-overrun error occurs when a newly received character is transferred into the rxdata holding register before the previous character is read
4	TOE	Read/Clear	Transmit overrun error. A transmit-overrun error occurs when a new character is written to the txdata holding register before the previous character is transferred into the shift register
5	TMT	Read/	Transmit empty. The TMT bit indicates the transmitter shift register's current state. When the shift register is in the process of shifting a character out the TXD pin, TMT is set to 0.
6	TRDY	Read	Transmit ready. When the txdata register is empty, it is ready for a new character, and TRDY is 1. When the txdata register is full,

			TRDY is 0.
7	RRDY	Read	Receive character ready. When the rxdata register is empty, it is not ready to be read and RRDY is 0. When a newly received value is transferred into the rxdata register, RRDY is set to 1. Reading the rxdata register clears the RRDY bit to 0.
8	E	Read/Clear	Exception. The E bit indicates that an exception condition occurred.
10	DCTS	Read/Clear	Change in clear to send (CTS) signal.
11	CTS	Read	Clear-to-send (CTS) signal.
12	EOP	Read	End of packet encountered. The EOP bit is set to 1 by one of the following events: ■ An EOP character is written to txdata ■ An EOP character is read from rxdata

❖ **Thanh ghi control:**

- Thanh ghi điều khiển bao gồm các bit riêng lẻ, mỗi bit điều khiển một khía cạnh hoạt động của lõi UART.
- Giá trị trong thanh điều khiển có thể được đọc bất cứ lúc nào. Mỗi bit trong thanh điều khiển kích hoạt ngắt (IRQ) cho một bit tương ứng trong thanh ghi status.
- Khi cả một bit trạng thái và bit bật/tắt ngắt tương ứng của nó đều là 1, UART sẽ tạo ra ngắt (IRQ).

Bit	Name	Access	Description
0	IPE	Read/Write	Enable interrupt for a parity error.
1	IFE	Read/Write	Enable interrupt for a framing error.
2	IBRK	Read/Write	Enable interrupt for a break detect.
3	IROE	Read/Write	Enable interrupt for a receiver overrun error.
4	ITOE	Read/Write	Enable interrupt for a transmitter overrun error.
5	ITMT	Read/Write	Enable interrupt for a transmitter shift register empty.
6	ITRDY	Read/Write	Enable interrupt for a transmission ready.
7	IRRDY	Read/Write	Enable interrupt for a read ready.
8	IE	Read/Write	Enable interrupt for an exception.
9	TRBK	Read/Write	Transmit break.
10	IDCTS	Read/Write	Enable interrupt for a change in CTS signal.
11	RTS	Read/Write	Request to send (RTS) signal.
12	IEOP	Read/Write	Enable interrupt for end-of-packet condition.

❖ **Ngắt UART:**

- UART sẽ xuất ra một tín hiệu ngắt IRQ khi:
- Mỗi điều kiện ngắt sẽ được thực hiện khi: có một bit liên quan trong thanh status và bit điều

khởi ngắt tương ứng trong thanh ghi điều khiển được cài đặt.

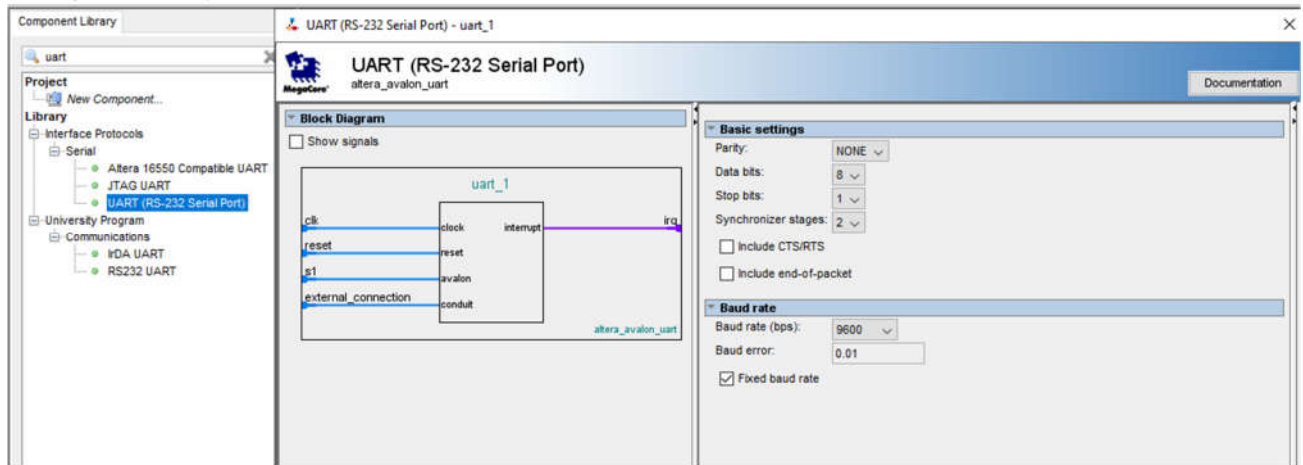
- Khi có bất kỳ điều kiện ngắt nào xảy ra, bit trạng thái ngắt liên quan sẽ được set là 1 và duy trì như vậy cho đến khi được xác nhận rõ ràng. Đầu ra IRQ được kích hoạt khi bất kỳ bit trạng thái nào được đặt trong khi bit điều khiển ngắt tương ứng là 1. Nios có thể kiểm tra ngắt IRQ bằng cách xóa thanh ghi trạng thái.
- Khi reset, tất cả các bit điều khiển ngắt đều được đặt thành 0; do đó, lỗi UART không thể kích hoạt IRQ cho đến khi một Nios cài đặt một hoặc nhiều bit điều khiển ngắt thành 1.

3. BÀI TẬP

Thực hiện hệ thống trên Qsys bao gồm các lỗi IP như sau:

- *On-chip memory – Ram*
- *Chip Nios II*
- *Clock 50*
- *JTAG UART*
- *UART*

Add thêm UART như sau, cài đặt **Baud Rate=9600**:



Sơ đồ hệ thống các lỗi IP trong Qsys:

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_0	Clock Source					
		clk_in	Clock Input	clk				
		clk_in_reset	Reset Input	Double-click to export				
		clk	Clock Output	Double-click to export	clk_0			
		clk_reset	Reset Output	Double-click to export				
☒		nios2_qsys_0	Nios II Processor					
		clk	Clock Input	Double-click to export	clk_0			
		reset_n	Reset Input	Double-click to export	[clk]			
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]			
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_0800	0x0001_0fff	
		custom_instruction_m...	Custom Instruction Master	Double-click to export				
☒		onchip_memory2_0	On-Chip Memory (RAM or ROM)					
		clk1	Clock Input	Double-click to export	clk_0			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0000_8000	0x0000_ffff	
		reset1	Reset Input	Double-click to export	[clk1]			
☒		jtag_uart_0	JTAG UART					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_1020	0x0001_1027	
☒		uart_0	UART (RS-232 Serial Port)					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_1000	0x0001_101f	
		external_connection	Conduit	uart_0_external_connecti...				

Code verilog Top-level:

```
1 module lab_uart(  
2     input CLOCK_50,  
3     input UART_RXD,  
4     output UART_TXD  
5 );  
6  
7     nios_sys u0 (  
8         .clk_clk (CLOCK_50), // clk.clk  
9         .uart_0_external_connection_rxd (UART_RXD), // uart_0_external_connection.rxd  
10        .uart_0_external_connection_txd (UART_TXD) // .txd  
11    );  
12  
13 endmodule
```

Viết code software sử dụng lõi IP Altera cung cấp sẵn sử dụng thư viện:

- altera_avalon_uart_regs.h
- altera_avalon_uart.h
- altera_avalon_uart.c

Hàm đọc data: **IORD_ALTERA_AVALON_UART_RXDATA(UART_BASE)**

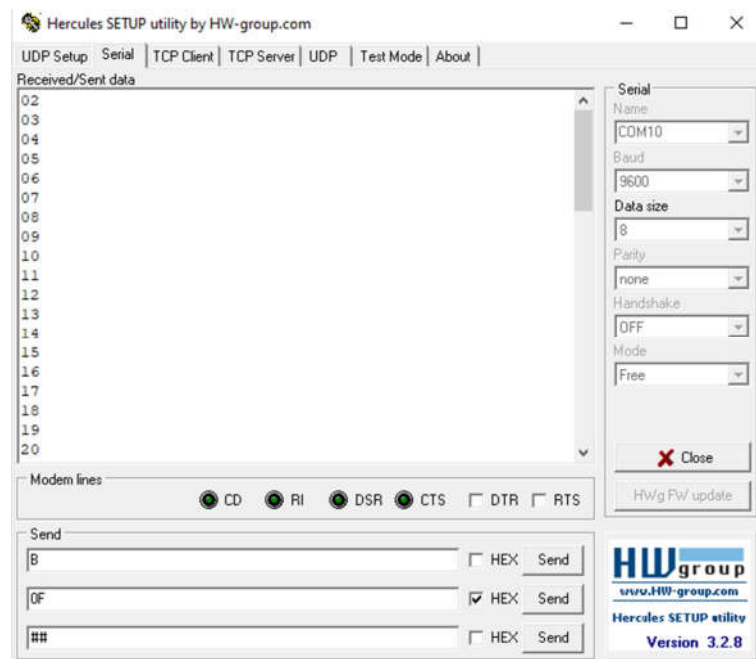
Hàm ghi data: **IOWR_ALTERA_AVALON_UART_TXDATA(UART_BASE, character)**

Code C hiển thị đếm từ 00 → 99:

```
#include "sys/alt_stdio.h"  
#include "system.h"  
#include <unistd.h>  
#include "stdio.h"  
#include "altera_avalon_uart_regs.h"  
  
char number[10]={'0','1','2','3','4','5','6','7','8','9'};  
  
int main()  
{  
    int i=0;  
    //alt_putstr("Hello from Nios II!\n");  
    printf("Start\n");  
  
    while(1)  
    {  
        for(i=0;i<100;i++)  
        {  
            IOWR_ALTERA_AVALON_UART_TXDATA(UART_0_BASE, number[i/10]);  
            usleep(1000); //wait to send data  
            IOWR_ALTERA_AVALON_UART_TXDATA(UART_0_BASE, number[i%10]);  
            usleep(1000); //wait to send data  
            IOWR_ALTERA_AVALON_UART_TXDATA(UART_0_BASE, '\n');  
            usleep(1000000);  
        }  
    }  
}
```

Thực hiện cài đặt Driver cho cáp USB-COM, và sử dụng phần mềm Hercules để truyền và nhận data

thông qua cổng COM, kết quả chạy như sau:



BÀI TẬP

Bài 1: 01 Button, UART và JTAG UART, khi nhấn Button thì đếm số lần nhấn nút từ 00 – 99 và truyền số này qua cả 2: UART và JTAG UART.

Bài 2: Thực hiện các lệnh điều khiển như sau.

- Khi nhận ký tự ‘S’ thì Nios truyền về String “Down”;
- Khi nhận ký tự ‘W’ thì Nios truyền về String “Up”;
- Khi nhận ký tự ‘A’ thì Nios truyền về String “Left”;
- Khi nhận ký tự ‘D’ thì Nios truyền về String “Right”;

Gợi ý:

- Hàm kiểm tra cờ báo data input trên thanh ghi Status :

```
if((IORD_ALTERA_AVALON_UART_STATUS(UART_0_BASE)&0x80) == 0x80)
```

- Ta cần khai báo biến kiểu Char để đọc data, ví dụ như sau:

```
Char rxData;
```

```
rxData = IORD_ALTERA_AVALON_UART_RXDATA(UART_0_BASE);
```

Code ví dụ cho hàm Transmit:

```
if(ALTERA_AVALON_UART_STATUS_TRDY_MSK==TRDY)// check if transmit ready
{
    IOWR_ALTERA_AVALON_UART_TXDATA(UART_BASE,mess->send);//send character
    Delay(600);//wait the end of transmission
}
```

Code ví dụ cho hàm Receive:

```
alt_u8 status;

status= IORD_ALTERA_AVALON_UART_STATUS(UART_BASE);

if(status & ALTERA_AVALON_UART_STATUS_RRDY_MSK){ //Check if UART is ready to
offer access to the RXDATA register
    buffer=IORD_ALTERA_AVALON_UART_RXDATA(UART_BASE);//receive character
    IOWR_ALTERA_AVALON_UART_STATUS(UART_BASE, 0); //clear the status bits again
}
```